

PERCEPTRON AND DECISION BOUNDARY

Md. Shaon Khan

Machine Learning / Deep Learning Foundations

Edition 1

সূচিপত্র (Table of Contents)

অধ্যায় ১ : Classification-এর ভূমিকা.....	3
অধ্যায় ২ : Feature Space ও Linear Classification.....	3
অধ্যায় ৩ : Perceptron-এর মূল কাঠামো.....	4
অধ্যায় ৪ : Weight, Bias, Weighted Sum ও Prediction.....	5
অধ্যায় ৫ : Decision Boundary ও তার Geometry.....	5
অধ্যায় ৬ : Weight Vector, Vector Form ও Transpose.....	6
অধ্যায় ৭ : Prediction থেকে Learning-এ যাওয়া.....	7
অধ্যায় ৮ : Perceptron Learning Rule ও Learning Trick	8
অধ্যায় ৯ : সম্পূর্ণসংখ্যাগত উদাহরণ.....	9
চূড়ান্ত Boundary যাচাই (Verification).....	9
একাধিক Decision Boundary সম্ভব.....	10
Learning Rate ও Epoch.....	10
অধ্যায় ১০ : Linearly Separable বনাম Non-Linearly Separable.....	11
XOR সমস্যা ও এর সীমাবদ্ধতা.....	11
অধ্যায় ১১ : Perceptron বনাম Decision Boundary — সারসংক্ষেপ	12
সাধারণ বিভ্রান্তি ও ভুল ধারণা	12
অধ্যায় ১২ : সংক্ষিপ্ত Formula Sheet.....	13
অধ্যায় ১৩ : চূড়ান্ত সারসংক্ষেপ (Final Revision).....	14
অধ্যায় ১৪ : সংক্ষিপ্ত Revision প্রশ্নোত্তর	14

অধ্যায় ১ : Classification-এর ভূমিকা (Introduction to Classification)

Classification হলো Machine Learning-এর একটি মৌলিক সমস্যা, যেখানে একটি Input-কে নির্দিষ্ট কোনো Class-এ assign করা হয়। যখন মোট Class সংখ্যা মাত্র দুইটি হয়, তখন তাকে বলা হয় Binary Classification, এবং এই দুইটি Class-কে সাধারণত Class 0 ও Class 1 নামে চিহ্নিত করা হয়।

প্রতিটি Input বর্ণনা করা হয় কিছু পরিমাপযোগ্য Feature দিয়ে। এরকম একটি সম্পূর্ণ বর্ণনাকে বলা হয় Data Point। প্রতিটি Data Point-এর সাথে যুক্ত থাকে একটি প্রকৃত Label বা Target, যা বলে দেয় Data Point-টি আসলে কোন Class-এর। অনেকগুলো Data Point-এর সংগ্রহকে বলা হয় Dataset।

সংক্ষিপ্তভাবে মূল ধারণাগুলো:

- **Input** — Classifier-কে যা দেওয়া হয়
- **Feature** — Input-এর একটি পরিমাপযোগ্য বৈশিষ্ট্য
- **Label / Target** — একটি Data Point-এর প্রকৃত Class
- **Dataset** — অনেক Data Point-এর সংগ্রহ

উদাহরণস্বরূপ, দুইটি Feature x_1 ও x_2 ব্যবহার করে যদি চারটি Data Point দেওয়া থাকে — যার মধ্যে দুইটি Class 0-এর এবং দুইটি Class 1-এর — তাহলে লক্ষ্য হলো এমন একটি নিয়ম বের করা, যা নতুন কোনো Data Point দেখেই বলে দিতে পারবে সেটি কোন Class-এ পড়বে। এই নিয়ম তৈরির জন্য প্রয়োজন একটি Classifier।

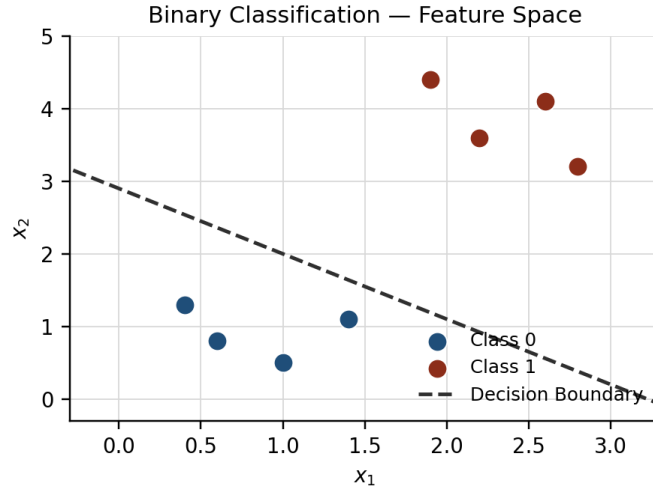
সবচেয়ে সরল ও ঐতিহাসিকভাবে গুরুত্বপূর্ণ একটি Classifier হলো Perceptron, যা এই বইয়ের মূল আলোচ্য বিষয়। পরবর্তী অধ্যায়গুলোতে ধাপে ধাপে দেখানো হবে কীভাবে Perceptron একটি Dataset থেকে শেখে এবং Class 0 ও Class 1-কে আলাদা করার জন্য একটি Decision Boundary তৈরি করে।

অধ্যায় ২ : Feature Space ও Linear Classification

প্রতিটি Data Point-কে তার Feature মানগুলো অনুযায়ী একটি স্থানে বসালে যে স্থান তৈরি হয়, তাকে বলা হয় Feature Space। একটি Feature থাকলে তা 1D Feature Space, দুইটি থাকলে 2D, তিনটি থাকলে 3D Feature Space। Feature সংখ্যাকে বলা হয় Dimension।

Feature Space-এ প্রতিটি Class-এর Data Point সাধারণত একটি নির্দিষ্ট অঞ্চলে (Classification Region) অবস্থান করে। যদি দুই Class-কে একটি সরলরেখা (2D-তে), সমতল (3D-তে), বা তার সাধারণ রূপ Hyperplane দিয়ে সম্পূর্ণভাবে আলাদা করা যায়, তাহলে সেই Dataset-কে বলা হয় Linearly Separable এবং এই বিভাজন পদ্ধতিকে বলা হয় Linear Classification।

এখানে "Linear" শব্দটি বোঝায় যে বিভাজনকারী সীমারেখাটি সম্পূর্ণ সরল — কোনো বাঁক বা বক্রতা নেই। নিচের চিত্রে x_1 ও x_2 নামক দুইটি Feature দিয়ে গঠিত একটি Feature Space দেখানো হয়েছে, যেখানে একটি সরলরেখা Class 0 ও Class 1-কে পরিষ্কারভাবে পৃথক করেছে।



চিত্র ২.১: Feature Space-এ Linear Classification — সরলরেখাটিই Decision Boundary।

লক্ষ্যণীয়, Feature, Data Point, Class এবং Boundary — এই চারটি ধারণা একে অপরের থেকে ভিন্ন: Feature হলো একটি একক পরিমাপ (যেমন x_1), Data Point হলো Feature-গুলোর একটি সম্পূর্ণ সেট, Class হলো একটি Data Point-এর পরিচয়, আর Boundary হলো সেইসীমারেখা যা বিভিন্ন Class-কে পৃথক করে। পরবর্তী অধ্যায়ে দেখানো হবে কীভাবে Perceptron ঠিক এই ধরনের একটি Linear Boundary নিজে থেকেই শিখে নিতে পারে।

অধ্যায় ৩ : Perceptron-এর মূল কাঠামো (Perceptron Fundamentals)

Perceptron হলো সবচেয়ে সরল Binary Classification model, যা একাধিক Input Feature নিয়ে একটি single Output তৈরি করে। এটি কৃত্রিম নিউরনের প্রথম দিককার গাণিতিক মডেল, এবং পরবর্তী সকল Neural Network-এর ভিত্তি মূলত এই ধারণা থেকেই এসেছে।

একটিমাত্র Feature x থাকলে Perceptron-এর হিসাব সবচেয়ে সরল রূপে লেখা যায়, যেখানে w হলো Weight এবং b হলো Bias:

$$z = wx + b$$

বাস্তব সমস্যায় সাধারণত একাধিক Feature থাকে। দুইটি Feature x_1 ও x_2 থাকলে প্রতিটির নিজস্ব Weight থাকে, এবং হিসাবটি হয়:

$$z = w_1x_1 + w_2x_2 + b$$

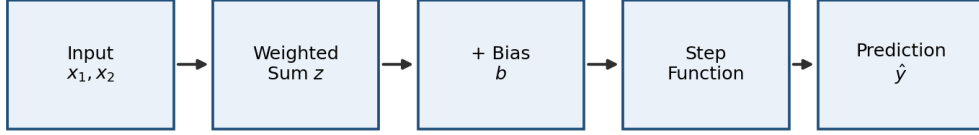
আরও বেশি Feature থাকলে প্রতিটি পদ আলাদাভাবে না লিখে Summation চিহ্ন ব্যবহার করে সংক্ষিপ্তভাবে লেখা হয়:

$$z = \sum_i w_i x_i + b$$

এই একই হিসাবকে Vector আকারেও লেখা যায়, যা অধ্যায় ৬-এ বিস্তারিতভাবে ব্যাখ্যা করা হয়েছে:

$$z = W^T X + b$$

Perceptron-এর সম্পূর্ণ প্রক্রিয়াটি পাঁচটি ধাপে ঘটে — প্রথমে Input নেওয়া হয়, তারপর সেগুলোর Weighted Sum বের করা হয়, এরপর তার সাথে Bias যোগ করা হয়, ফলাফল z -এর ওপর Step Function প্রয়োগ করা হয়, এবং শেষে একটি Binary Prediction $\hat{y}$ পাওয়া যায়। নিচের চিত্রে এই প্রবাহ (flow) দেখানো হলো।



চিত্র ৩.১: Perceptron আর্কিটেকচার — Input থেকে Prediction পর্যন্ত প্রবাহ।

অধ্যায় ৪ : Weight, Bias, Weighted Sum ও Prediction

Weight

প্রতিটি Feature-এর একটি করে Weight থাকে, যা নির্দেশ করে সেই Feature চূড়ান্ত সিদ্ধান্তে কতটা প্রভাব ফেলবে। Weight ধনাত্মক (Positive) হলে সংশ্লিষ্ট Feature-এর মান বাড়লে z -ও বাড়ে; Weight ঋণাত্মক (Negative) হলে ঠিক তার বিপরীত ঘটে। Weight-এর পরম মান যত বড় হয়, সেই Feature-এর প্রভাব তত বেশি হয়, আর Weight শূন্যের কাছাকাছি হলে সেই Feature-এর প্রভাব প্রায় থাকে না।

Bias

Bias হলো এমন একটি অতিরিক্ত প্যারামিটার, যা কোনো Feature-এর ওপর নির্ভর না করেই z -এর মান পরিবর্তন করতে পারে। Bias শূন্য ($b = 0$) হলে Decision Boundary সবসময় মূলবিন্দু $(0, 0)$ দিয়ে যেতে বাধ্য হয়। Bias পরিবর্তন করলে Decision Boundary সরে যায় (shift), কিন্তু তার দিক (orientation) অপরিবর্তিত থাকে — এটি Boundary-কে ঘোরায় না, শুধু সরায়।

Weighted Sum — একটি সংখ্যাগত উদাহরণ

ধরা যাক $w_1 = 2$, $w_2 = -1$, $b = 1$, এবং একটি Data Point $x_1 = 3$, $x_2 = 2$ । তাহলে Weighted Sum হবে:

$$z = (2)(3) + (-1)(2) + 1 = 6 - 2 + 1 = 5$$

Step Function ও Prediction

Weighted Sum z -কে সরাসরি Prediction হিসেবে ব্যবহার করা যায় না, কারণ Binary Classification-এর Output হতে হবে কেবল 0 অথবা 1। এই রূপান্তরটি করে Step Function, যা z -কে 0-এর সাথে তুলনা করে একটি Threshold হিসেবে ব্যবহার করে:

$$\hat{y} = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

এখানে যদি True Label y এবং Prediction $\hat{y}$ একই হয়, তাহলে Prediction সঠিক; না হলে তা একটি Error। এই তুলনাই পরবর্তী অধ্যায়ে Perceptron-এর Learning Rule-এর ভিত্তি তৈরি করবে।

অধ্যায় ৫ : Decision Boundary ও তার Geometry

Decision Boundary হলো Feature Space-এর সেই সীমারেখা, যেখানে Perceptron-এর সিদ্ধান্ত Class 0 থেকে Class 1-এ পরিবর্তিত হয়। যেহেতু z -এর মান অনুযায়ী Prediction নির্ধারিত হয়, তাই এই Boundary সংজ্ঞায়িত হয় ঠিক সেইসব বিন্দু দিয়ে যেখানে z -এর মান 0, কারণ এখানেই সিদ্ধান্ত এক Class থেকে আরেক Class-এ পরিবর্তিত হয়:

$$z = 0$$

দুই-Feature ক্ষেত্রে (x_1, x_2) এই সমীকরণটি হয়:

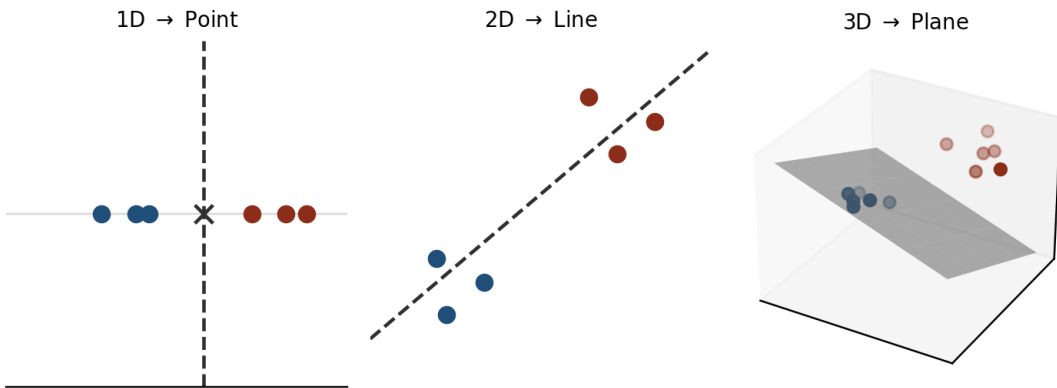
$$w_1x_1 + w_2x_2 + b = 0$$

এখান থেকে তিনটি সম্ভাব্য অবস্থা তৈরি হয়:

$$z > 0 \Rightarrow \text{Class 1}, \quad z < 0 \Rightarrow \text{Class 0}, \quad z = 0 \Rightarrow \text{Decision Boundary}$$

Dimension-এর সম্পর্ক

একটি গুরুত্বপূর্ণ Geometric নিয়ম হলো — Input-এর Dimension যত হয়, Decision Boundary-এর Dimension তার চেয়ে ঠিক একটি কম হয়। কারণ Boundary নিজে Feature Space-এর একটি "কাটাকাটি অংশ" (slice), যা সম্পূর্ণ স্থানের চেয়ে এক ধাপ কম Dimension-এর হতে বাধ্য।



চিত্র ৫.১: 1D → Point, 2D → Line, 3D → Plane — Dimension কমে যাওয়ার সম্পর্ক।

এই নিয়মটি যেকোনো সংখ্যক Dimension-এর জন্য সাধারণভাবে প্রযোজ্য:

$$N\text{-dimensional input} \rightarrow (N - 1)\text{-dimensional Decision Boundary}$$

অধ্যায় ৬ : Weight Vector, Vector Form ও Transpose

দুইটি Feature থাকলে সব Weight একত্রে একটি Column Vector হিসেবে এবং সব Feature একত্রে আরেকটি Column Vector হিসেবে লেখা যায়:

$$W = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \quad X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

কিন্তু Matrix Multiplication-এর নিয়ম অনুযায়ী দুইটি Column Vector সরাসরি গুণ করা যায় না। তাই W-কে Transpose করে একটি Row Vector-এ রূপান্তর করা হয়:

$$W^T = [w_1 \quad w_2]$$

এরপর এই Row Vector-কে Column Vector X দিয়ে গুণ করলে একটি Scalar পাওয়া যায়, যা ঠিক অধ্যায় ৩-এর scalar হিসাবের সমান:

$$W^T X = [w_1 \quad w_2] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = w_1 x_1 + w_2 x_2$$

কেন Transpose ব্যবহার করা হয়?

Matrix Multiplication তখনই সংজ্ঞায়িত, যখন প্রথম Matrix-এর Column সংখ্যা দ্বিতীয় Matrix-এর Row সংখ্যার সমান হয়। W ও X উভয়ই (2×1) আকারের Column Vector হওয়ায় সরাসরি গুণ করা সম্ভব নয়। W-কে Transpose করে (1×2) আকারে আনলে এবং তা X-এর (2×1) আকারের সাথে গুণ করলে ফলাফল হয় (1×1) , অর্থাৎ একটি একক Scalar — যা ঠিক আমাদের প্রয়োজনীয় z।

Weight Vector-এর Geometric অর্থ

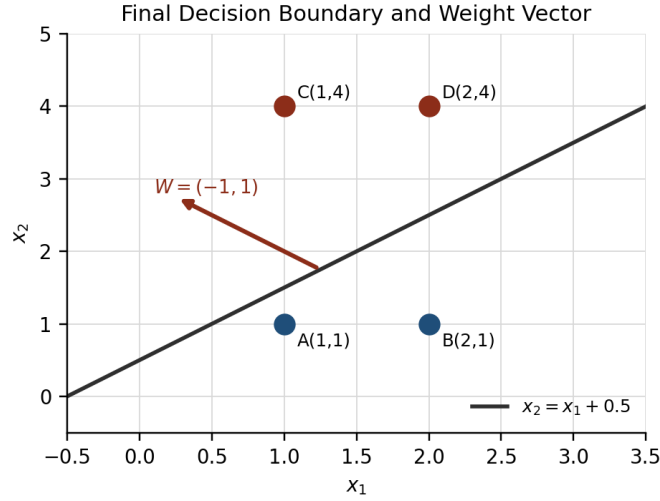
Weight Vector W-এর একটি গুরুত্বপূর্ণ Geometric বৈশিষ্ট্য হলো, এটি সবসময় Decision Boundary-এর সাথে লম্ব (Perpendicular) থাকে এবং সবসময় Class 1 -এর দিকে নির্দেশ করে:

$$W \perp \text{Decision Boundary}$$

উদাহরণস্বরূপ, পরবর্তী অধ্যায়ের সংখ্যাগত উদাহরণে প্রাপ্ত চূড়ান্ত Decision Boundary এবং তার সাথে সংশ্লিষ্ট Weight Vector নিচে দেখানো হলো:

$$-x_1 + x_2 - 0.5 = 0 \quad \Longleftrightarrow \quad x_2 = x_1 + 0.5$$

$$W = (-1, 1)$$



চিত্র ৬.১: Decision Boundary $x_2 = x_1 + 0.5$ এবং তার সাথে লম্ব Weight Vector $W = (-1, 1)$

অধ্যায় ৭ : Prediction থেকে Learning-এ যাওয়া

Perceptron শেখে তার ভুল থেকে। এই শেখার প্রক্রিয়া বোঝার জন্য প্রথমে True Label y এবং Prediction $\hat{y}$ -এর মধ্যে পার্থক্য বোঝা প্রয়োজন। y হলো একটি Data Point-এর প্রকৃত Class, আর $\hat{y}$ হলো Perceptron-এর দেওয়া Prediction। এই দুইটি মিলে গেলে Prediction সঠিক, না মিললে তা একটি Error।

True y	Prediction $\hat{y}$	ফলাফল (Result)
0	0	Correct
1	1	Correct
0	1	Wrong
1	0	Wrong

এখান থেকেই Perceptron Learning-এর কেন্দ্রীয় নিয়মটি তৈরি হয়:

$$y = \hat{y} \Rightarrow \text{No Update}, \quad y \neq \hat{y} \Rightarrow \text{Update}$$

অর্থাৎ, Prediction সঠিক হলে Weight ও Bias-এ কোনো পরিবর্তন আনা হয় না; কেবল Prediction ভুল হলেই Update করা হয়। এই সরল নিয়মটিই পরবর্তী অধ্যায়ের Perceptron Learning Rule-এর ভিত্তি।

অধ্যায় ৮ : Perceptron Learning Rule ও Learning Trick

যখন Prediction ভুল হয়, তখন Perceptron তার Weight এবং Bias নিচের নিয়ম অনুযায়ী Update করে:

$$w_{\text{new}} = w_{\text{old}} + \eta(y - \hat{y})x$$

$$b_{\text{new}} = b_{\text{old}} + \eta(y - \hat{y})$$

এখানে প্রতিটি চিহ্নের অর্থ:

- **w (নতুন), w (পুরনো)** — Update-এর পরে ও আগে Weight-এর মান — উপরের সমীকরণে যথাক্রমে subscript new ও old দিয়ে দেখানো হয়েছে
- **η (eta)** — Learning Rate — Update-এর পদক্ষেপের আকার নিয়ন্ত্রণ করে
- **$y, \hat{y}$** — True Label এবং Prediction

- x — সংশ্লিষ্ট Feature-এর মান
- b (নতুন), b (পুরনো) — Update-এর পরে ও আগে Bias-এর মান

দুইটি সম্ভাব্য Error-এর ক্ষেত্রে

Case 1 — $y = 1, \hat{y} = 0$: প্রকৃত Class 1 হলেও Perceptron 0 Predict করেছে। এক্ষেত্রে $(y - \hat{y}) = 1$ হয়, ফলে Update Weight ও Bias-কে বাড়িয়ে দেয়, যাতে পরবর্তীতে একই Input-এর জন্য z -এর মান বেড়ে যায় এবং Prediction Class 1 -এর দিকে স্থানান্তরিত হয়।

Case 2 — $y = 0, \hat{y} = 1$: প্রকৃত Class 0 হলেও Perceptron 1 Predict করেছে। এক্ষেত্রে $(y - \hat{y}) = -1$ হয়, ফলে Update Weight ও Bias-কে কমিয়ে দেয়, যাতে z -এর মান কমে যায় এবং Prediction Class 0 -এর দিকে স্থানান্তরিত হয়।

এই সম্পূর্ণ প্রক্রিয়াটিকে একত্রে বলা হয় Perceptron Learning Trick:

Wrong Prediction $\rightarrow$ Error $\rightarrow$ Update $W, b \rightarrow$ Boundary Changes $\rightarrow$ Repeat

অধ্যায় ৯ : সম্পূর্ণ সংখ্যাগত উদাহরণ (Complete Numerical Example)

এই অধ্যায়ে আমরা একটি সম্পূর্ণ Dataset-এর উপর Perceptron Learning Rule ধাপে ধাপে প্রয়োগ করবো। Dataset-টি নিচে দেওয়া হলো:

Point	x_1	x_2	y
A	1	1	0
B	2	1	0
C	1	4	1
D	2	4	1

প্রাথমিক মান: $w_1 = 0, w_2 = 0, b = 0$, এবং Learning Rate $\eta = 0.5$ । Point-গুলো এই ক্রমে প্রক্রিয়া করা হবে: $A \rightarrow C \rightarrow D \rightarrow B$ । প্রতিটি ধাপে Step Function ব্যবহার করে Prediction বের করা হবে, এবং প্রয়োজনে Weight ও Bias Update করা হবে।

Point	z -এর হিসাব	$\hat{y}$	$y=\hat{y}$?	নতুন W, b
A	$(0)(1)+(0)(1)+0 = 0$	1	না — Update	$W=(-0.5, -0.5), b=-0.5$
C	$(-0.5)(1)+(-0.5)(4)-0.5 = -3.0$	0	না — Update	$W=(0, 1.5), b=0$
D	$(0)(2)+(1.5)(4)+0 = 6.0$	1	হ্যাঁ — কোনো Update নেই	$W=(0, 1.5), b=0$
B	$(0)(2)+(1.5)(1)+0 = 1.5$	1	না — Update	$W=(-1, 1), b=-0.5$

প্রতিটি ধাপের ব্যাখ্যা:

Point A: প্রকৃত Label 0 হলেও $z \geq 0$ হওয়ায় Prediction হয় 1, যা ভুল। তাই Weight ও Bias-কে Update করে Class 0 -এর দিকে ঠেলে দেওয়া হয়:

$$W = (-0.5, -0.5), \quad b = -0.5$$

Point C: প্রকৃত Label 1 হলেও নতুন Weight দিয়ে হিসাব করলে z ঋণাত্মক হয়, ফলে Prediction হয় 0, যা ভুল। তাই আবার Update করা হয়, এবার Class 1 -এর দিকে:

$$W = (0, 1.5), \quad b = 0$$

Point D: বর্তমান Weight ও Bias দিয়েই z বড় ধনাত্মক হয় এবং Prediction সঠিকভাবে 1 আসে। যেহেতু $y = \hat{y}$, তাই কোনো Update প্রয়োজন হয় না — Weight ও Bias অপরিবর্তিত থাকে।

Point B: প্রকৃত Label 0 হলেও বর্তমান Weight দিয়ে z ধনাত্মক আসে, ফলে Prediction ভুলভাবে 1 হয়। তাই শেষবারের মতো Update করা হয়, যার ফলে চূড়ান্ত Weight ও Bias পাওয়া যায়:

$$W = (-1, 1), \quad b = -0.5$$

এইভাবে সম্পূর্ণ একটি Epoch (Dataset-এর একবার সম্পূর্ণ পাস) শেষে চূড়ান্ত Decision Boundary পাওয়া যায়:

$$-x_1 + x_2 - 0.5 = 0 \quad (\text{i.e., } x_2 = x_1 + 0.5)$$

চূড়ান্ত Boundary যাচাই (Final Boundary Verification)

চূড়ান্ত মান $W = (-1, 1)$, $b = -0.5$ ব্যবহার করে সবগুলো Data Point পুনরায় যাচাই করা যাক:

Point	z -এর হিসাব	$\hat{y}$	সঠিক?
A(1,1)	$-(1)+(1)-0.5 = -0.5$	0	হ্যাঁ
B(2,1)	$-(2)+(1)-0.5 = -1.5$	0	হ্যাঁ
C(1,4)	$-(1)+(4)-0.5 = 2.5$	1	হ্যাঁ
D(2,4)	$-(2)+(4)-0.5 = 1.5$	1	হ্যাঁ

সবগুলো Point সঠিকভাবে শ্রেণিবিন্যাসিত হওয়ায় দেখা যাচ্ছে যে এই Decision Boundary সম্পূর্ণ Dataset-এর জন্য কাজ করছে:

$$\text{Training Accuracy} = 100\%$$

একাধিক Decision Boundary সম্ভব

একটি গুরুত্বপূর্ণ পর্যবেক্ষণ হলো, একটি Linearly Separable Dataset-এর জন্য একাধিক সঠিক Decision Boundary থাকতে পারে। আমাদের উদাহরণে A ও B (Class 0) থেকে C ও D (Class 1)-কে আলাদা করার জন্য একাধিক ভিন্ন সরলরেখা ব্যবহার করা সম্ভব — Perceptron কেবল এমন একটি Boundary খুঁজে বের করে, যা Training Data-কে সঠিকভাবে পৃথক করতে পারে।

Perceptron কিন্তু স্বভাবতই এই বিষয়গুলো নিশ্চিত করে না: Maximum Margin, উভয় Class থেকে সমান দূরত্ব, বা সবচেয়ে Robust Boundary। তাই আমাদের প্রাপ্ত চূড়ান্ত Boundary হয়তো A ও B-এর কাছাকাছি এবং C, D থেকে দূরে অবস্থিত — কিন্তু তা সত্ত্বেও এটি একটি সম্পূর্ণ সঠিক Decision Boundary, কোনো ব্যর্থতা নয়।

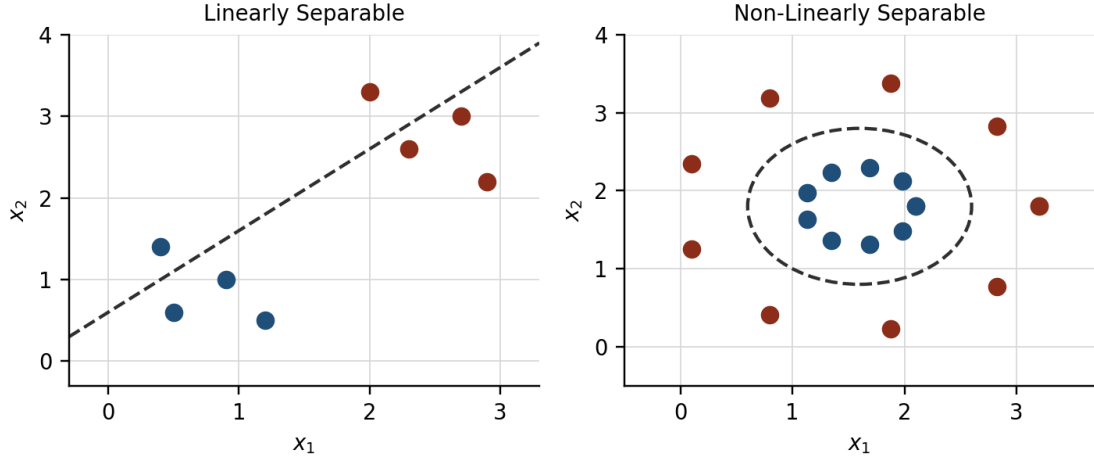
Learning Rate ও Epoch

Learning Rate (η) নির্ধারণ করে প্রতিটি Update-এ Weight ও Bias কতটা পরিবর্তিত হবে। ছোট η দিয়ে Update ধীরে ধীরে ও স্থিতিশীলভাবে হয়, কিন্তু শেখার প্রক্রিয়া ধীর হতে পারে; বড় η দিয়ে Update দ্রুত হয়, কিন্তু Decision Boundary অতিরিক্ত পরিবর্তিত হয়ে যেতে পারে।

Epoch বলতে বোঝায় সম্পূর্ণ Dataset-এর একবার সম্পূর্ণ পাস। Linearly Separable Data-এর জন্য একাধিক Epoch-এর পরে এমন একটি অবস্থায় পৌঁছানো যায়, যেখানে কোনো Point-এই আর ভুল হয় না — তখন Training থামানো যায়।

অধ্যায় ১০ : Linearly Separable বনাম Non-Linearly Separable

একটি Dataset Linearly Separable হয়, যদি একটিমাত্র Linear Decision Boundary দিয়ে তার সব Class-কে ভুল ছাড়াই আলাদা করা যায়। বিপরীতে, একটি Dataset Non-Linearly Separable হয়, যদি কোনো একক Linear Boundary দিয়ে তা সম্ভব না হয় — তখন Class-গুলো এমনভাবে মিশে থাকে যে একটি সরলরেখা, সমতল বা Hyperplane কখনোই তাদের নিখুঁতভাবে পৃথক করতে পারে না।



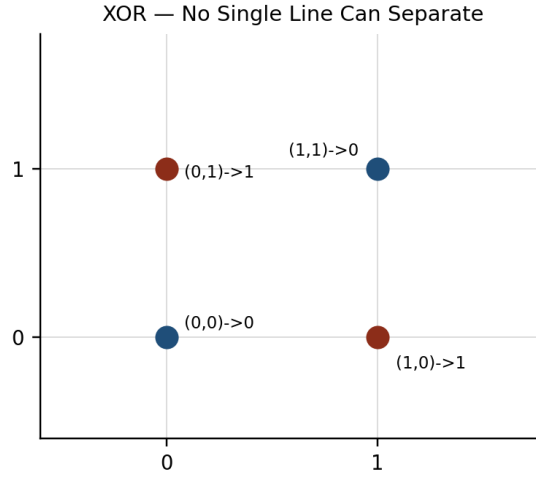
চিত্র ১০.১: বাম দিকে একটি Linearly Separable Dataset; ডান দিকে একটি Non-Linearly Separable Dataset, যেখানে একটি Class অন্যটির দ্বারা ঘেরাও হয়ে আছে।

একটি Single Perceptron কেবল Linear Decision Boundary তৈরি করতে পারে, তাই এটি শুধুমাত্র Linearly Separable Dataset-এর জন্যই সঠিকভাবে কাজ করে। Non-Linearly Separable Dataset-এর ক্ষেত্রে Single Perceptron কখনোই সব Point-কে সঠিকভাবে শ্রেণিবিন্যাস করতে পারবে না, তা যতই Epoch চালানো হোক না কেন।

XOR সমস্যা ও এর সীমাবদ্ধতা

XOR (Exclusive OR) একটি ক্লাসিক উদাহরণ, যা Non-Linearly Separable Data-এর সবচেয়ে পরিচিত দৃষ্টান্ত। এর সত্যতা সারণি (Truth Table) নিচে দেওয়া হলো:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



চিত্র ১০.২: XOR-এর চারটি Point — কোনো একক সরলরেখা দিয়ে Class 0 ও 1 কে পৃথক করা সম্ভব নয়।

লক্ষ্য করলে দেখা যায়, Class 1 -এর দুইটি Point $(0, 1)$ ও $(1, 0)$ কর্ণ-বরাবর অবস্থিত, এবং Class 0 -এর দুইটি Point $(0, 0)$ ও $(1, 1)$ বিপরীত কর্ণে অবস্থিত। এই বিন্যাসের কারণে কোনো একক সরলরেখা কখনোই সবগুলো Point-কে সঠিক Class-এ আলাদা করতে পারে না।

Single Perceptron cannot solve XOR

এই সীমাবদ্ধতাই পরবর্তীতে Multi-Layer Neural Network-এর উদ্ভাবনের একটি বড় কারণ হয়ে দাঁড়ায়, যেখানে একাধিক Perceptron-সদৃশ Unit একত্রে কাজ করে Non-Linear Decision Boundary তৈরি করতে সক্ষম হয়। এই বইয়ের পরিধি Single Perceptron ও তার Decision Boundary পর্যন্ত সীমাবদ্ধ থাকায় Multi-Layer Network-এর বিস্তারিত আলোচনা এখানে অন্তর্ভুক্ত করা হয়নি।

অধ্যায় ১১ : Perceptron বনাম Decision Boundary — একটি সারসংক্ষেপ

নিচের সারণিতে এই বইয়ের মূল ধারণাগুলোর মধ্যে সম্পর্ক সংক্ষিপ্তভাবে দেখানো হলো:

ধারণা (Concept)	অর্থ (Meaning)
Perceptron	Classification model নিজে
Weight	শেখার যোগ্য (Learnable) প্যারামিটার
Bias	শেখার যোগ্য (Learnable) প্যারামিটার
Weighted Sum	Input-গুলোর সাংখ্যিক সমষ্টি (z)
Step Function	z -কে Binary Output-এ রূপান্তর করে
Prediction	Model-এর দেওয়া Output ($\hat{y}$)
Decision Boundary	যে সীমারেখা Class-গুলোকে আলাদা করে
Learning Rule	Weight ও Bias Update করার নিয়ম

এই সম্পর্কগুলোর মধ্যে সবচেয়ে গুরুত্বপূর্ণ দুইটি হলো:

$$W, b \rightarrow \text{Decision Boundary}$$

Wrong Prediction $\rightarrow$ Update $W, b \rightarrow$ Boundary Changes

সাধারণ বিভ্রান্তি ও ভুল ধারণা (Common Confusions)

- Weight বনাম Bias — Weight একটি নির্দিষ্ট Feature-এর সাথে যুক্ত, কিন্তু Bias কোনো Feature-নির্ভর নয়।
- Weighted Sum (z) বনাম Prediction ($\hat{y}$) — z একটি ধারাবাহিক (Continuous) সংখ্যা, কিন্তু $\hat{y}$ কেবল 0 অথবা 1।
- True Label (y) বনাম Prediction ($\hat{y}$) — y হলো প্রকৃত সত্য, $\hat{y}$ হলো Model-এর অনুমান।
- Decision Boundary বনাম Classification Region — Boundary হলো সীমারেখা নিজে ($z=0$), আর Region হলো তার দুই পাশের অঞ্চল ($z>0$ বা $z<0$)।
- Input Dimension বনাম Boundary Dimension — N -মাত্রিক Input-এর Boundary হয় $(N-1)$ -মাত্রিক।
- Transpose কেন প্রয়োজন — কারণ দুইটি Column Vector সরাসরি গুণ করা যায় না; একটিকে Row Vector-এ আনতে হয়।
- কেন $z=0$ Decision Boundary সংজ্ঞায়িত করে — কারণ এখানেই Prediction এক Class থেকে আরেক Class-এ পরিবর্তিত হয়।
- সঠিক Prediction-এ কেন Update হয় না — কারণ $(y - \hat{y}) = 0$ হলে Update Term নিজেই শূন্য হয়ে যায়।
- একটি সঠিক Boundary সবসময় Maximum-Margin Boundary নয় — Perceptron কেবল সঠিক বিভাজন নিশ্চিত করে, সর্বোত্তম বিভাজন নয়।

অধ্যায় ১২ : সংক্ষিপ্ত Formula Sheet

Revision-এর সুবিধার্থে বইয়ের সবগুলো গুরুত্বপূর্ণ সমীকরণ একত্রে নিচে দেওয়া হলো।

Weighted Sum

$$z = \sum_i w_i x_i + b$$

Vector Form

$$z = W^T X + b$$

Step Function

$$\hat{y} = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Decision Boundary

$$W^T X + b = 0$$

2D Decision Boundary

$$w_1 x_1 + w_2 x_2 + b = 0$$

Weight Update

$$w_{\text{new}} = w_{\text{old}} + \eta(y - \hat{y})x$$

Bias Update

$$b_{\text{new}} = b_{\text{old}} + \eta(y - \hat{y})$$

Dimension Rule

N -dimensional input $\rightarrow (N - 1)$ -dimensional Decision Boundary

অধ্যায় ১৩ : চূড়ান্ত সারসংক্ষেপ (Final Revision)

সম্পূর্ণ বইয়ের ধারণাপ্রবাহ একত্রে দেখানো হলো:

Input Features $\rightarrow$ Weighted Sum $\rightarrow$ Bias $\rightarrow z \rightarrow$ Step Function $\rightarrow$ Prediction

Prediction $\rightarrow$ Compare with True Label $\rightarrow$ Error

Error $\rightarrow$ Update $W, b \rightarrow$ Decision Boundary Changes $\rightarrow$ Repeat

এবং সবচেয়ে গুরুত্বপূর্ণ উপসংহার দুইটি:

Linearly Separable Data $\rightarrow$ Single Perceptron can learn

Non-Linearly Separable Data $\rightarrow$ Single Perceptron cannot solve

অধ্যায় ১৪ : সংক্ষিপ্ত Revision প্রশ্নোত্তর

1. Perceptron কী?

একটি সরল Binary Classification model, যা Weighted Sum ও Step Function ব্যবহার করে Prediction তৈরি করে।

2. Binary Classification কী?

যখন Classification-এর ফলাফল কেবল দুইটি সম্ভাব্য Class (0 বা 1) হতে পারে।

3. Feature কী?

একটি Input-এর একটি পরিমাপযোগ্য বৈশিষ্ট্য।

4. Weight কী?

একটি Feature-এর প্রভাব নিয়ন্ত্রণকারী শেখার-যোগ্য প্যারামিটার।

5. Bias কী?

Feature-নির্ভর নয় এমন একটি অতিরিক্ত প্যারামিটার, যা Boundary-কে সরাসরি সাহায্য করে।

6. Weighted Sum কী?

সব Feature ও তার Weight-এর গুণফলের সমষ্টি, Bias সহ।

7. Step Function কী?

Weighted Sum z -কে 0 বা 1 -এ রূপান্তরকারী Threshold Function।

8. Decision Boundary কী?

যে সীমারেখায় Prediction এক Class থেকে অন্য Class-এ পরিবর্তিত হয়।

9. $z = 0$ কেন Decision Boundary?

কারণ এখানেই Prediction 0 থেকে 1 -এ পরিবর্তিত হয়।

10. 2D Input-এর Boundary কেন 1D?

কারণ N -মাত্রিক Input-এর Boundary সবসময় $(N-1)$ -মাত্রিক হয়।

11. $W^T X$ -এ Transpose কেন?

দুইটি Column Vector সরাসরি গুণ করা যায় না; একটিকে Row Vector-এ আনতে হয়।

12. Perceptron কখন Update করে?

যখন Prediction ভুল হয়, অর্থাৎ $y \neq \hat{y}$ ।

13. কখন Update করে না?

যখন Prediction সঠিক হয়, অর্থাৎ $y = \hat{y}$ ।

14. Learning Rate কী?

Update-এর পদক্ষেপের আকার নিয়ন্ত্রণকারী একটি সংখ্যা (η)।

15. Epoch কী?

সম্পূর্ণ Dataset-এর একবার সম্পূর্ণ পাস।

16. Linearly Separable Data কী?

যে Data একটিমাত্র Linear Boundary দিয়ে সম্পূর্ণভাবে আলাদা করা যায়।

17. Single Perceptron কেন XOR সমাধান করতে পারে না?

কারণ XOR-এর Class-গুলো কর্ণ-বরাবর বিন্যস্ত, যা কোনো একক সরলরেখা দিয়ে পৃথক করা সম্ভব নয়।

18. Weight/Bias ও Decision Boundary-এর সম্পর্ক কী?

W ও b একসাথে Decision Boundary-এর অবস্থান ও দিক নির্ধারণ করে।